

O sistema HiGTree/HiGFlow: inventário dos módulos existentes

Relatório técnico

Setembro de 2026

Resumo

Inventário do que existe no código do HiGTree/HiGFlow: 75 arquivos-fonte e cerca de 76 mil linhas, distribuídas entre a biblioteca de malha, o solver, os adaptadores compartilhados e dezessete exemplos executáveis. Os números vem de contagem direta da árvore.

O levantamento torna visíveis três coisas que a leitura do código por partes não mostra: a entrada e saída é o maior módulo isolado do sistema, com 20 % do HiGFlow; os modelos reológicos ocupam três vezes mais linhas que a infraestrutura do solver; e a combinatória de físicas está mais *declarada* que realizada — dois dos cinco acoplamentos têm menos de trinta linhas e apenas despacham.

Este é um inventário, e não uma avaliação: a presença de um módulo não diz que ele esteja correto, verificado ou em uso.

Sumário

1	Introdução	1
1.1	O que é o sistema	1
1.2	A linhagem	1
1.3	O que este relatório é, e o que não é	2
1.4	O grupo e a produção sobre o sistema	2
1.5	Sobre as referências, e o limite desta busca	3
2	HiGTree: a camada de malha	4
3	HiGFlow: o solver	4
3.1	Um módulo de passo por tipo de escoamento	4
3.2	E os acoplamentos entre eles	5
3.3	Modelos constitutivos	5
3.4	Discretizações, por eixo	5
3.5	Tratamento de interface	6
3.6	Entrada e saída: o módulo mais improvável	6
4	Exemplos e código compartilhado	6
4.1	Os dezessete casos	6
4.2	<code>examples-common</code> : o que os exemplos compartilham	7
4.3	Integração com o <code>t8code</code>	7
5	A estrutura descrita e a estrutura de hoje	7
5.1	HiGTree: o diagrama resistiu	7
5.2	HiGFlow: o esqueleto resistiu; o volume, não	8
5.3	O que a comparação sugere	8

6	O projeto de extensão CPU/GPU, confrontado com a medida	9
6.1	As três premissas de desempenho	9
6.2	O que foi medido	9
6.3	Três consequências	9
6.4	O que isto sugere para a ordem do trabalho	10
6.5	O alcance desta seção, e os seus limites	10
7	O que o inventário mostra	11
7.1	Onde estão as linhas	11
7.2	O que este inventário não diz	11

1 Introdução

1.1 O que é o sistema

O HiGFlow é um simulador de escoamentos incompressíveis construído sobre o HiGTree, uma biblioteca de malhas em árvore **não graduada** com interpolação sem malha. A fundação do método está em Sousa et al. [27]: diferenças finitas em árvore onde células vizinhas podem diferir de mais de um nível de refino, com a interpolação por mínimos quadrados ponderados cobrindo o que o estêncil regular não alcança.

A escolha de não exigir graduação é o que separa o sistema da maioria dos códigos de AMR, e ela aparece em toda parte: no `domain.c`, que monta estênceis sem supor vizinhança uniforme; no `wls.c`, que é a interpolação que torna isso possível; e no custo — cada avaliação fora do estêncil regular resolve um sistema local.

1.2 A linhagem

O sistema não começou em 2019. Ele é a terceira geração de uma linha de simuladores do mesmo grupo:

- **GENSMAC** [29] — marcadores e células para superfície livre em domínios gerais, 1994;
- **Freeflow** [9] — o sistema integrado 3D que embrulhou o GENSMAC em ambiente de simulação, 2000, com a versão axissimétrica em [28];
- **o front-tracking do próprio grupo** [14] — a combinação de rastreamento e captura de frente para multifluido 3D, 2004, que é o antecedente direto do módulo de front-tracking descrito adiante;
- **HiGTree/HiGFlow** [27] — a malha em árvore não graduada, 2019.

Ler a lista na ordem explica escolhas do código que de outro modo pareceriam arbitrárias — a presença simultânea de VOF e front-tracking, por exemplo, reproduz a coexistência que [14] já propunha.

1.3 O que este relatório é, e o que não é

É um **inventário**: o que existe no código, em quantas linhas, e qual o papel de cada módulo. Os números vem de contagem direta da árvore, não de estimativa.

Não é uma avaliação de correção. Dizer que um módulo existe não é dizer que ele está certo, nem que está verificado, nem que é usado. Onde houver verificação conhecida, ela está marcada; onde não houver, a ausência também.

1.4 O grupo e a produção sobre o sistema

Esta seção foi reescrita a partir da apresentação do próprio grupo (`Apresentacao_HiG-Flow.pptx`), que traz a lista de colaboradores e a produção. Cada referência foi depois conferida no Crossref pelo DOI.

	nome	vínculo	
	Adenilso da Silva Simão	ICMC	
	Fabício Simeoni de Sousa	ICMC	
	Leandro Franco de Souza	ICMC	
	Alexandre Afonso	FEUP (Portugal)	
	Célio B. P. Fernandes	FEUP (Portugal)	
	Luís Ferrás	FEUP (Portugal)	
	Aristeu da Silveira Neto	UFU	
	Rafael Figueiredo	UFU	
	Jonas Ansoni	UFLA	
	Manuel Silvino Batalha	UFPA	
Vinte e dois colaboradores, em seis instituições.	Wesley de Sousa Bezerra	UFMS	
	Ricardo Serfaty	Petrobras	
	Hugo Sanchez	ICMC	
	Juliana Bertoco	ICMC	pós-
	Camila Lages	ICMC	
	Juniormar Organista	ICMC	
	Aquisson Silva	ICMC	
	Fabiano Ruano Neto	ICMC	
	Alexandre Miziara	ICMC	
	Rameyli Godoi	ICMC	
	Pedro Vinicius	ICMC	
	Kainã Tureso	ICMC	

A coluna *papel* distingue orientação concluída de em andamento onde o dado permite. Os anos entre parênteses vem dos registros de tese conferidos, e não da apresentação.

Trinta publicações, e o que elas cobrem. A apresentação declara 13 artigos em periódicos internacionais, 4 trabalhos em anais do SBMAC e 13 teses e dissertações da USP. As áreas acompanham os módulos inventariados adiante: eletro-osmose [6], modelos integrais KBKZ [1, 2], gPTT [3], tixotropia [10], suspensões [11], VOF tridimensional [13] e a própria arquitetura de malhas hierárquicas [8, 12, 20].

As teses, e uma discrepância de contagem que não resolvi. A apresentação declara **treze**. A busca por autor no banco da USP localizou **catorze** trabalhos do ICMC sobre os temas do sistema — os treze mais a dissertação de Miziara [22], sobre VOF com viscoelasticidade, cujo autor também assina um dos trabalhos do SBMAC.

Não forcei o número para bater. As duas candidatas a ficar de fora são as mais antigas — Figueiredo [15] e Bertoco [4], de 2011 e 2012 —, que podem ter usado os antecessores do sistema em vez dele próprio; os metadados não dizem. Decidir isso exige saber o que cada uma usou, e quem orientou saberá. Listo as catorze e registro a dúvida.

Por que elas importam para este inventário. Elas são a **documentação de projeto que o código não tem**: vários módulos correspondem um a um a uma tese, e quem precisar entender uma escolha de implementação encontra ali o que os comentários não dizem.

- **Interface e multifásico** — Silva [25] (PLIC-VOF) e [26]; Lages [18, 19]; Figueiredo [15, 16]; Miziara [22] (VOF com viscoelasticidade);
- **Viscoelástico** — Bertoco [4, 5] (KBKZ-PSM); Leiva [21] (modelos generalizados); Godoi [17] (viscosidade variável); Ruano Neto [24];
- **Eletro-osmose** — Bezerra [7];
- **Verificação** — Organista [23], sobre soluções manufaturadas. É a única da lista cujo assunto é *verificar* o sistema, e não acrescentar-lhe física.

A correspondência mais direta: o módulo `hig-flow-step-viscoelastic-variable-viscosity.c`, com 2435 linhas, é o assunto de [17]; o `hig-flow-step-shear-thickening-suspension.c`, com 3413, corresponde a [11].

1.5 Sobre as referências, e o limite desta busca

As entradas da bibliografia foram conferidas contra o registro do editor via API do Crossref, e não contra memória.

Uma lição sobre o alcance da busca automática. A primeira versão desta seção localizou **uma** única dissertação — a de Godoi — e registrou que certamente havia outras, sem poder nomeá-las. O Crossref indexa metadados, e apenas um dos treze títulos contém “HiGFlow”.

Com os nomes dos alunos em mãos, a busca por *autor* no mesmo índice achou as treze em minutos. **Não faltava base de dados: faltava a chave de busca.** Fica o registro porque a conclusão anterior — “está fora do alcance das ferramentas” — era falsa, e o que a desfez foi uma informação que qualquer pessoa do grupo teria.

E aqui o limite precisa ser dito, porque ele é grande. O Crossref indexa metadados — título, autores, resumo —, e não texto integral. Uma tese que *use* o HiGFlow sem nomeá-lo no título não aparece nessa busca. Dado que o sistema tem módulos para tixotropia, bandas de cisalhamento, suspensões, eletro-osmose e viscoelasticidade integral — cada um com milhares de linhas, e cada um o tipo de assunto que rende uma dissertação —, é quase certo que existam outras. Elas não estão aqui porque **não foram encontradas**, e não porque não existam.

Listar referências prováveis sem conferi-las seria pior que omiti-las: um relatório de sistema é consultado justamente para saber o que citar. O caminho para completar esta seção é uma busca de texto integral no repositório de teses da USP, que está fora do alcance das ferramentas usadas aqui — ou, mais direto, a indicação dos nomes por quem os conheça. O arquivo `referencias.bib` traz um encaixe com o modelo de entrada e o comando de conferência, para que acrescentá-las seja mecânico.

Os contribuidores do repositório. O histórico Git cobre de setembro de 2020 a setembro de 2026, com 358 *commits*. Os nomes que aparecem são Antonio Castelo Filho, Juniormar Organista, Daniel Garcia, Johnatas e Pedro Vinícius. Isso é autoria de *commit*, que não coincide necessariamente com autoria dos módulos nem com as teses — registro o dado, não a inferência.

2 HiGTree: a camada de malha

Trinta e três fontes, 15 869 linhas. É a biblioteca sobre a qual tudo o mais se apóia, e ela não sabe nada de Navier–Stokes.

módulo	linhas	papel
domain.c	3 316	domínio serial: células, facetas, estênceis, mapeadores
lbal.c	1 979	balanceamento de carga e partição das árvores
higtreet-io.c	1 826	leitura do <i>.amr</i> , escrita de VTK
pdomain.c	1 799	domínio <i>distribuído</i> : franja, sincronização, tipos MPI
higtreet.c	843	a árvore em si: refino, busca, geometria de célula
higtreet-iterator.c	832	iteradores sobre células e facetas
wls.c	624	mínimos quadrados ponderados, base da interpolação
hig-mesh-snapshot.c	213	instantâneo da malha, para o oráculo de faceta

A divisão que mais importa é domain contra pdomain. O primeiro é o domínio serial — células, facetas, estênceis, mapeadores. O segundo é o distribuído, e é onde vivem a *franja* (as cópias de células de outro *rank*), a sincronização e os tipos derivados de MPI. Juntos são 5 115 linhas, quase um terço da biblioteca: a distribuição é o assunto mais caro dela.

Solvers lineares: cinco implementações atrás de uma interface. PETSc, HYPRE, SOR, centralizado e um de depuração que simplesmente grava o sistema. O de PETSc é o usado na prática. Existe também um `solver-petsc.camila.c`, variante paralela do principal — duas cópias de 576 e 555 linhas que divergiram na primeira mudança de protocolo.

wls.c é o coração do método. Sem os mínimos quadrados ponderados, a árvore não graduada não fecha: é ele que produz valor onde o estêncil regular não existe. As 624 linhas são pequenas para o peso que carregam.

3 HiGFlow: o solver

Quarenta e dois fontes, 54 457 linhas — três vezes e meia a HiGTree. A maior parte disso são *modelos*, não infraestrutura.

3.1 Um módulo de passo por tipo de escoamento

O `flowtype` da configuração seleciona qual passo roda:

flowtype	módulo	linhas
NEWTONIAN	hig-flow-step.c	1 640
GENERALIZED_NEWTONIAN	step-generalized-newtonian.c	778
MULTIPHASE	step-multiphase.c	2 513
VISCOELASTIC	step-viscoelastic.c	1 915
VISCOELASTIC_INTEGRAL	step-viscoelastic-integral.c	2 301
VISCOELASTIC_VAR_VISCOSITY	step-viscoelastic-variable-viscosity.c	2 435
SHEAR_BANDING	step-viscoelastic-shear-banding.c	3 225
ELASTOVISCOPLASTIC	step-elastoviscoplastic.c	1 680
SUSPENSIONS	step-shear-thickening-suspension.c	3 413

São 19 900 linhas só em passos de escoamento. O maior deles — suspensões, com 3 413 — é maior que `domain.c`, o núcleo da HiGTree.

3.2 E os acoplamentos entre eles

módulo	linhas	acopla
<code>step-electroosmotic.c</code>	2 624	eletrocinética ao escoamento
<code>step-multiphase-electroosmotic.c</code>	2 172	duas fases + eletrocinética
<code>step-multiphase-viscoelastic.c</code>	903	duas fases + viscoelasticidade
<code>step-electroosmotic-viscoelastic.c</code>	31	esqueleto: despacha para os dois
<code>step-multiphase-electroosmotic-viscoelastic.c</code>	28	esqueleto: despacha para os três

Os dois últimos têm 31 e 28 linhas: são despachantes, não implementações. A combinatória de físicas está declarada mais do que realizada, e vale saber disso antes de contar com ela.

3.3 Modelos constitutivos

família	modelos
viscoelástico diferencial	UCM, Oldroyd-B, Giesekus, LPTT, GPTT
viscoelástico integral	KBKZ, KBKZ fracionário
elastoviscoplastico	Oldroyd-B/Bingham, Oldroyd-B/Herschel–Bulkley, LPTT/Bingham
tixotrópico	BMP, BMP com solvente
suspensões	engrossamento por cisalhamento

3.4 Discretizações, por eixo

eixo	opções
tempo	Euler explícito, RK2, RK3, Euler semi-implícito, Crank–Nicolson, BDF2
convecção (1a ordem)	central, primeira ordem, segunda ordem
convecção (alta ordem)	MCU, CUBISTA e demais esquemas de limitação
espacial	segunda e quarta ordem
projeção	não incremental, incremental

O produto cartesiano dessas opções é grande, e nada garante que todas as combinações tenham sido exercitadas — esta é uma lista do que o código *aceita*, não do que foi verificado.

3.5 Tratamento de interface

família	módulos	papel
VOF — advecção	2	2D e 3D, transporte da fração volumétrica
VOF — reconstrução	4	PLIC 2D/3D, ELVIRA, Mehta
VOF — curvatura	5	altura de fluido (HF), diferenças finitas, Shirani, 9 células
front-tracking	1	polilinha lagrangeana ordenada (núcleo standalone)
fronteira imersa	1	marcadores por posse, DMSwarm, espalhamento com franja
remalhamento	1	transferência por posição, consciente de MPI

Onze módulos de VOF, contra um de front-tracking e um de fronteira imersa. A assimetria conta a história do sistema: o VOF é o caminho maduro, com variantes de reconstrução e de curvatura acumuladas ao longo de anos; o front-tracking é recente e tem uma implementação só.

3.6 Entrada e saída: o módulo mais improvável

`hig-flow-io.c` tem **10 972 linhas** — 20 % de todo o HiGFlow, e mais que a soma de `domain.c` e `pdomain.c`. Ele lê as configurações em YAML, grava e relê o estado, e escreve VTK. Cada

combinação de `flowtype` com dimensão tem o seu par de leitura/escrita, e é daí que vem o tamanho.

É o módulo que ninguém planeja e que cresce sozinho.

4 Exemplos e código compartilhado

4.1 Os dezessete casos

exemplo	o que exercita
<code>example2d_Newt</code>	Navier–Stokes newtoniano
<code>example2d_Newt_contraction</code>	contração 4:1
<code>example2d_Oldroyd</code>	viscoelástico diferencial
<code>example2d_Gptt</code>	GPTT
<code>example2d_KBKZ</code>	viscoelástico integral
<code>example2d_BMP</code>	tixotropia
<code>example2d_ElectroOsmotic</code>	eletro-osmose
<code>example2d_VOF</code>	duas fases por VOF
<code>example2d_VOF_Oldroyd</code>	duas fases viscoelásticas
<code>example2d_VOF_Gptt</code>	duas fases GPTT
<code>example2d_FrontTracking</code>	duas fases por front-tracking
<code>example2d_FronteiraImersa</code>	corpo rígido imerso
<code>example2d_DynamicMeshAdapt</code>	adaptação dinâmica de malha
<code>example2d_SchaeferTurek</code>	<i>benchmark</i> de cilindro
<code>example3d_SchaeferTurek</code>	o mesmo em 3D
<code>example3d_lid_driven</code>	cavidade 3D
<code>example3d_complex</code>	geometria complexa 3D

Cada exemplo é um programa próprio, com o seu `main` e o seu `Makefile`, que linka os objetos de `higflow/src` de que precisa. Não há biblioteca única do HiGFlow: há uma coleção de objetos e dezessete formas de combiná-los.

4.2 `examples-common`: o que os exemplos compartilham

Cinco mil seiscentas e cinquenta e oito linhas fora de `src`, e é onde vive o código que é demais para um exemplo e de menos para a biblioteca:

- `utilities-2d.c` (2024 linhas) — condições de contorno e iniciais compartilhadas;
- `front-tracking.c` (995) — o adaptador entre o núcleo lagrangeano e o solver;
- `malha-adaptativa.c` (789) — critério de refino e reconstrução, compartilhado entre VOF e front-tracking;
- `malha-t8.cxx` (442) — as fontes de malha do `t8code`, compiladas apenas quando `T8CODE` é passado ao `make`;
- `vof-geometry-2d.c` e `eo-droplet-2d.c` — geometrias iniciais específicas.

4.3 Integração com o `t8code`

O `t8code` entra como *fonte de malha* opcional, com dupla adesão: em tempo de compilação (`T8CODE=<prefixo>`) e em tempo de execução (`HIGFLOW_MALHA=`). Sem as duas, o caminho é o `mtree`, lendo o `.amr`.

Duas limitações, ambas verificadas no código:

1. o caminho **multifásico** não consulta a fonte de malha — `higflow_partition_domain_multiphase` lê o `.amr` sempre. A fronteira imersa funciona com `t8code` porque é monofásica e passa por outra função;
2. as fontes do `t8code` **recusam** `.amr` multinível, com mensagem explícita (“recusar é melhor que aproximar”). Como o critério adaptativo escreve exatamente isso, integrar o `t8code` ao caso adaptativo exige uma fonte que *leia* a lista de refino — que o `higio_amr_info` carrega e entrega, e que hoje é descartada em favor só da caixa envolvente.

5 A estrutura descrita e a estrutura de hoje

A apresentação do grupo traz dois diagramas de módulos, um por camada. Vale confrontá-los com a árvore: um diagrama que envelheceu em silêncio é pior que a ausência dele, porque continua orientando quem chega.

5.1 HiGTree: o diagrama resistiu

no diagrama	hoje	
Solver	5 implementações: PETSc, HYPRE, SOR, centralizado, dep.	
Interpoladores	<code>wls.c</code> , mínimos quadrados ponderados	
Domínio	<code>domain.c</code> + <code>pdomain.c</code> , 5 115 linhas	
Interfaces	<code>hig-mesh-contract.h</code> , <code>hig-mesh-snapshot.c</code>	
Estrutura geom. (euleriana)	<code>higtree.c</code> , <code>higtree-iterator.c</code>	
Estrutura geom. (lagrangeana)	não existe na HiGTree	MIGROU

Cinco dos seis blocos continuam exatos, com nomes de arquivo reconhecíveis. A camada de malha é estável — o que se espera de uma biblioteca de infraestrutura, e o que justifica ela ser uma camada separada.

A exceção é a malha lagrangeana, e ela migrou de camada. O diagrama a coloca na HiGTree, ao lado da euleriana. Na árvore de hoje não há estrutura lagrangeana na HiGTree: o que existe é `point-cloud.c`, e o próprio cabeçalho dele diz o que é — “um conjunto crescente de pontos, sem estrutura espacial própria”, o acumulador que as buscas por caixa preenchem antes de os pontos virarem suporte de interpolação. É infraestrutura da interpolação, não malha de partículas.

As estruturas lagrangeanas de fato estão **no HiGFlow**: `hig-flow-front-tracking.c`, com a polilinha ordenada de marcadores, e `hig-flow-fronteira-imersa.c`, com os marcadores por posse sobre `DMSwarm`. São dois modelos de distribuição diferentes — um replicado, outro repartido — e nenhum deles é genérico o bastante para descer à camada de malha. A migração parece consequência disso, e não descuido.

5.2 HiGFlow: o esqueleto resistiu; o volume, não

no diagrama	hoje	estado
Main Program	17 main próprios, um por exemplo	expandido
IO	<code>hig-flow-io.c</code> , 10 972 linhas	igual, e enorme
IC	<code>hig-flow-ic.c</code>	igual
BC	<code>hig-flow-bc.c</code>	igual
Step	12 módulos: 9 <code>flowtype</code> + 5 acoplamentos	multiplicado
Discretization	<code>hig-flow-discret.c</code>	igual
Eval	<code>hig-flow-eval.c</code>	igual
Kernel	<code>hig-flow-kernel.c</code>	igual
—	16 módulos de interface (VOF, FT, FI, remalha)	NÃO está no diagrama
—	<code>terms</code> , <code>res</code> , <code>timestep</code> , <code>linear-algebra</code>	NÃO está no diagrama

Os oito blocos do diagrama existem todos, com os nomes de arquivo correspondentes — `io`, `ic`, `bc`, `step`, `discret`, `eval`, `kernel`. Como esqueleto, ele está certo. O que mudou foi o que cabe dentro de cada caixa, e em duas direções:

Step deixou de ser um bloco. São doze módulos — nove tipos de escoamento e cinco acoplamentos —, somando quase 24 mil linhas. Um retângulo chamado “Step” num diagrama e doze arquivos de mil a três mil e quatrocentas linhas são objetos diferentes, e a diferença importa para quem for estender o sistema: acrescentar um modelo não é mexer no *Step*, é escrever um irmão dele.

E há uma família inteira fora do diagrama. Dezesesseis módulos de tratamento de interface — treze de VOF, mais front-tracking, fronteira imersa e remalhamento — não têm caixa nenhuma. Junto com `terms`, `res`, `timestep` e `linear-algebra`, são cerca de dez mil linhas que o diagrama não prevê.

Não é crítica ao diagrama: ele descreve o *fluxo* de uma simulação — ler, iniciar, aplicar contorno, avançar, discretizar, avaliar — e nesse eixo continua correto. O tratamento de interface não é uma etapa do fluxo; é uma dimensão transversal, e diagramas de fluxo não a mostram bem.

5.3 O que a comparação sugere

- **a separação em duas camadas se sustentou.** Seis anos depois do artigo fundador, a fronteira entre malha e física continua onde foi posta, e a camada de baixo quase não se moveu;
- **o crescimento foi todo em modelos**, e por *multiplicação* de irmãos — um módulo por `flowtype`, um por acoplamento, um por variante de VOF. É o padrão que explica também o tamanho do `hig-flow-io.c`: cada combinação pede o seu par de leitura e escrita;
- **um diagrama de fluxo não mostra o eixo transversal.** Se o diagrama for atualizado algum dia, o tratamento de interface pede uma representação própria, e não mais uma caixa na fila.

6 O que o inventário mostra

6.1 Onde estão as linhas

camada	arquivos	linhas	papel
HiGTree — malha	33	15 869	árvore, domínio, partição, E/S, solvers lineares
HiGFlow — solver	42	54 457	Navier–Stokes e os modelos constitutivos
examples-common	12	5 658	adaptadores compartilhados entre exemplos
exemplos	17	—	casos executáveis, 2D e 3D

Três observações que a contagem torna difíceis de ignorar.

A E/S é o maior módulo isolado do sistema. `hig-flow-io.c` sozinho tem 20 % do HiGFlow e mais linhas que o domínio serial e o distribuído somados. A causa é estrutural: cada `flowtype` $\times$ dimensão $\times$ operação tem o seu par de leitura e escrita, e o crescimento é multiplicativo.

Os modelos dominam o solver. Dezenove mil e novecentas linhas em passos de escoamento, contra cerca de seis mil de infraestrutura (`kernel`, `discret`, `bc`, `ic`, `eval`, `terms`). O sistema é, em volume, uma coleção de modelos reológicos com um solver de Navier–Stokes dentro.

A combinatória está declarada mais do que realizada. Há nove `flowtype`, cinco acoplamentos, doze modelos constitutivos, seis discretizações temporais e duas projeções. O produto é enorme, e dois dos acoplamentos têm menos de trinta linhas. Uma lista de capacidades lida como catálogo superestima o que está exercitado.

6.2 O t8code: o que se pretendia, o que se fez, e por que não escala aqui

O t8code entrou no sistema com um objetivo declarado: ser uma opção de particionamento paralelo ao lado do MTree, motivada pela expectativa de que o MTree limite o número de células e a qualidade do balanceamento em cenários de refinamento heterogêneo. A ideia é boa e a biblioteca é a certa para isso: florestas de octrees distribuídas, balanceamento por curvas de preenchimento de espaço, refino e engrossamento entre processos.

	aspecto	estado
O que foi feito.	gancho nos exemplos	existe: <code>malha_t8_instala</code> , cinco fontes de malha
	adesão	dupla: <code>T8CODE=</code> na compilação e <code>HIGFLOW_MALHA=</code> na execução
	caminho monofásico	consulta a fonte; a fronteira imersa usa e funciona
	caminho multifásico	não consulta — lê o <code>.amr</code> sempre
	malha multinível	recusada pelas fontes (<code>numlevels != 1</code>)
	procedência da malha no log	impressa sempre; denuncia o pedido ignorado

A infraestrutura está montada e funciona onde é consultada: a fronteira imersa, que é monofásica, roda com t8code hoje. O que não se completou foi a ligação no caminho **multifásico** — onde vivem o VOF e o front-tracking —, que lê o `.amr` incondicionalmente. E há um segundo impedimento atrás do primeiro: as fontes recusam malha multinível, com a mensagem “recusar é melhor que aproximar”, e o critério adaptativo escreve exatamente isso.

E por que, mesmo completada, ela não daria escalabilidade neste sistema. Esta é a parte que a medida responde, e a resposta é desconfortável.

O remalhamento inteiro — escrita do critério mais reconstrução do domínio — custa entre 0,06 % e 0,10 % do tempo de execução, medido em $n_p = 1, 2, 4$ e 8. **Trocar o mecanismo de**

particionamento otimiza um milésimo da corrida. E a eficiência paralela medida é de 41 % em dois processos e 22 % em oito, com $n_p = 2$ chegando a ser mais lento que a execução serial: o gargalo não é a maquinaria de malha.

A razão estrutural está em três propriedades do sistema, todas verificadas no código e nenhuma delas no *backend* de particionamento:

1. o caso tem **uma única árvore**, e o laço de leitura reparte *arquivos* entre *processos*: com um arquivo só, um processo materializa a malha global **inteira em memória** antes de o balanceador a repartir. Nenhum particionador conserta um teto de memória que ocorre *antes* dele;
2. **higflow_reconstroi_dominio reconstrói o domínio do zero** a cada remalhamento, em vez de adaptar no lugar — o oposto do modo de operação que torna o t8code escalável;
3. o critério de refino passa por **arquivo escrito por um processo só**, com barreira, e relido por todos. É um trecho serial proporcional à malha global, a cada remalhamento.

O t8code é escalável porque adapta a floresta *no lugar* e reparticiona sem que descrição global alguma seja materializada. Encaixá-lo atrás da interface atual — que entrega e recebe uma descrição global de malha — pede dele exatamente aquilo que ele existe para evitar. A opção passaria a estar disponível, a partição ficaria melhor balanceada, e as três propriedades acima continuariam de pé.

O que de fato entregaria. Adaptação dinâmica distribuída: critério avaliado localmente por cada processo, floresta adaptada no lugar, reparticionamento pela curva de preenchimento, e o `.amr` reduzido a formato de *arranque* em vez de via de remalhamento. Isso é reescrever o caminho de remalhamento, e não ligar um gancho — uma decisão de outra ordem de grandeza, que deve ser tomada sabendo disso.

Duas ressalvas, e elas são grandes. Primeira: tudo isto foi medido num caso de cerca de 9 400 células. Num problema grande o equilíbrio muda — o solver escalaria melhor, e o teto de memória do item 1 passaria a morder, que é precisamente o que a adaptação distribuída resolve. **A conclusão não é “o t8code não serve”**: é que, neste regime, a justificativa para integrá-lo não pode ser o custo do remalhamento.

Segunda: que esses três gargalos *dominem* é inferência, não medida. O que foi medido é que o remalhamento é negligenciável e que a eficiência é baixa; atribuir a segunda aos três itens é leitura do código. Uma inferência do mesmo tipo já caiu neste trabalho — a de que a escrita serial do `.amr` seria o gargalo crítico, quando ela é 3 % de um custo que é 0,1 % do total.

6.3 O que este inventário não diz

- **nada sobre correção.** Contagem de linhas não é verificação, e a presença de um módulo não diz se ele foi validado contra oráculo algum;
- **nada sobre uso.** Não foi medido quais módulos os exemplos de fato exercitam, e há sinais de duplicação — duas variantes do solver de PETSc, dois de `solver.c`;
- **nada sobre paralelismo.** Quais módulos funcionam em $n_p > 1$ é outra pergunta, e a resposta varia por módulo;
- **a pós-graduação associada está incompleta**, pelas razões da introdução: a busca alcança metadados, não texto integral.

Referências

- [1] J. Bertoco, M. S. B. Araújo, R. T. Leiva, H. A. C. Sánchez, and A. Castelo. Numerical simulation of KBKZ integral constitutive equations in hierarchical grids. *Applied Sciences*, 11(11):4875, 2021.
- [2] J. Bertoco, A. Castelo, L. L. Ferrás, and C. Fernandes. Numerical simulation of three-dimensional free surface flows using the K-BKZ-PSM integral constitutive equation. *Polymers*, 15(18):3705, 2023.
- [3] J. Bertoco, R. T. Leiva, L. L. Ferrás, A. M. Afonso, and A. Castelo. Development length of fluids modelled by the gPTT constitutive differential equation. *Applied Sciences*, 11(21):10352, 2021.
- [4] Juliana Bertoco. Simulação numérica de equações de conservação usando esquemas *upwind*. Master’s thesis, Universidade de São Paulo, São Carlos, 2012.
- [5] Juliana Bertoco. *Solução numérica do modelo constitutivo KBKZ-PSM para escoamentos com superfícies livres*. PhD thesis, Universidade de São Paulo, São Carlos, 2017.
- [6] W. D. S. Bezerra, A. Castelo, and A. M. Afonso. Numerical study of electro-osmotic fluid flow and vortex formation. *Micromachines*, 10(12):796, 2019.
- [7] Wesley de Souza Bezerra. *Estudo numérico de escoamento viscoelástico e eletroosmótico com potenciais dependentes do tempo*. PhD thesis, Universidade de São Paulo, São Carlos, 2019.
- [8] A. Castelo, A. M. Afonso, and W. De Souza Bezerra. A hierarchical grid solver for simulation of flows of complex fluids. *Polymers*, 13(18):3168, 2021.
- [9] A. Castelo F., M. F. Tomé, C. N. L. César, S. McKee, and J. A. Cuminato. Freeflow: an integrated simulation system for three-dimensional free surface flows. *Computing and Visualization in Science*, 2(4):199–210, 2000.
- [10] H. A. Castillo-Sánchez, J. Bertoco, M. S. B. de Araujo, and A. Castelo. Numerical simulation of thixotropic–viscoelastic models for structured fluids in hierarchical grids. *Computers & Fluids*, 266:106045, 2023.
- [11] H. A. Castillo-Sánchez, J. Gillissen, R. Lange, and A. Castelo. Computational fluid dynamics simulations of suspensions of spherical particles using tensorial constitutive models. *Computers & Fluids*, 299:106704, 2025.
- [12] H. A. Castillo-Sánchez, L. F. Souza, and A. Castelo. Numerical simulation of rheological models for complex fluids using hierarchical grids. *Polymers*, 14(22):4958, 2022.
- [13] A. T. G. da Silva, C. Fernandes, J. Organista, L. Souza, and A. Castelo. Validation of HiGFlow software for simulating two-phase flows with a 3D geometric volume of fluid algorithm. *Mathematics*, 11(18):3900, 2023.
- [14] F. S. de Sousa, N. Mangiavacchi, L. G. Nonato, A. Castelo, M. F. Tomé, V. G. Ferreira, J. A. Cuminato, and S. McKee. A front-tracking/front-capturing method for the simulation of 3D multi-fluid flows with free surfaces. *Journal of Computational Physics*, 198(2):469–499, 2004.
- [15] Rafael Alves Figueiredo. Estudos de métodos numéricos para a simulação de escoamentos viscoelásticos com superfície livre. Master’s thesis, Universidade de São Paulo, São Carlos, 2011.

- [16] Rafael Alves Figueiredo. *Simulação numérica de escoamentos viscoelásticos multifásicos complexos*. PhD thesis, Universidade de São Paulo, São Carlos, 2016.
- [17] Rameyli Godoi. Implementação de modelos viscoelásticos com viscosidade variável no sistema HiGFlow. Master’s thesis, Universidade de São Paulo, São Carlos, 2022.
- [18] Camila Faria Afonso Lages. Métodos numéricos para escoamentos multifásicos em malhas hierárquicas. Master’s thesis, Universidade de São Paulo, São Carlos, 2016.
- [19] Camila Faria Afonso Lages. *Método de projeção multiescala e malhas hierárquicas para escoamentos incompressíveis modelados pelas equações de Navier–Stokes*. PhD thesis, Universidade de São Paulo, São Carlos, 2023.
- [20] R. T. Leiva, L. L. Ferrás, A. Castelo, et al. A generalisation of the integral maxwell model: the gK-BKZ model. *Meccanica*, 59(3):363–384, 2024.
- [21] Rosalía Taboada Leiva. *Desenvolvimento e implementação numérica de modelos viscoelásticos generalizados e o estudo de fluidos complexos*. PhD thesis, Universidade de São Paulo, São Carlos, 2022.
- [22] Gustavo Alexandre Sousa Miziara. Aplicação do método VOF para simulação dos escoamentos bifásicos com viscoelasticidade. Master’s thesis, Universidade de São Paulo, São Carlos, 2020.
- [23] Juniormar Organista. *Verificação de código de simulação numérica de escoamentos de fluidos viscoelásticos através do método de soluções manufaturadas*. PhD thesis, Universidade de São Paulo, São Carlos, 2025.
- [24] Fabiano Ruano Neto. *Numerical analysis and computational implementations of non-Newtonian flow constitutive equations*. PhD thesis, Universidade de São Paulo, São Carlos, 2025.
- [25] Aquisson Theyllon Gomes da Silva. Desenvolvimento da técnica PLIC-VOF no sistema HiG-Flow para simulação de escoamentos bifásicos. Master’s thesis, Universidade de São Paulo, São Carlos, 2020.
- [26] Aquisson Theyllon Gomes da Silva. *Modelagem de interfaces em escoamentos bifásicos de fluidos viscoelásticos usando o software HiG-Flow*. PhD thesis, Universidade de São Paulo, São Carlos, 2025.
- [27] F. S. Sousa, C. F. Lages, J. L. Ansoni, A. Castelo, and A. Simão. A finite difference method with meshless interpolation for incompressible flows in non-graded tree-based grids. *Journal of Computational Physics*, 396:848–866, 2019.
- [28] M. F. Tomé, A. Castelo, J. Murakami, J. A. Cuminato, R. Minghim, M. C. F. Oliveira, N. Mangiavacchi, and S. McKee. Numerical simulation of axisymmetric free surface flows. *Journal of Computational Physics*, 157(2):441–472, 2000.
- [29] M. F. Tomé and S. McKee. GENSMAC: A computational marker and cell method for free surface flows in general domains. *Journal of Computational Physics*, 110(1):171–186, 1994.